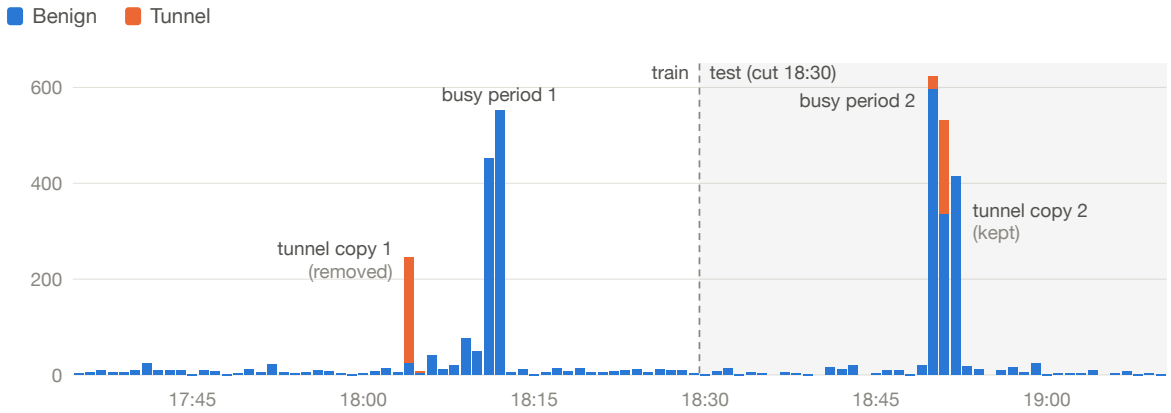
DNS tunnel detection: data pipeline

One capture of normal DNS plus a hidden tunnel session. The pipeline prepares it for an Isolation Forest. JM2030 Cybersecurity, Group 3, case 1.

DNS queries	Duration	Tunnel session (shown twice)	Features
3,645	95 min	1	8

The capture, minute by minute



The tunnel session was recorded twice. Copy 2 is an exact replay of copy 1. Benign traffic is quiet, with two busy periods.

How the pipeline runs

dns.log + labels
3,645 rows, joined by row_id

→

prepare.py
Wire-byte names, parent domain, one tunnel copy, split

→

features.py
8 numeric features per query, causal

→

models
Isolation Forest and baselines

The setup and why

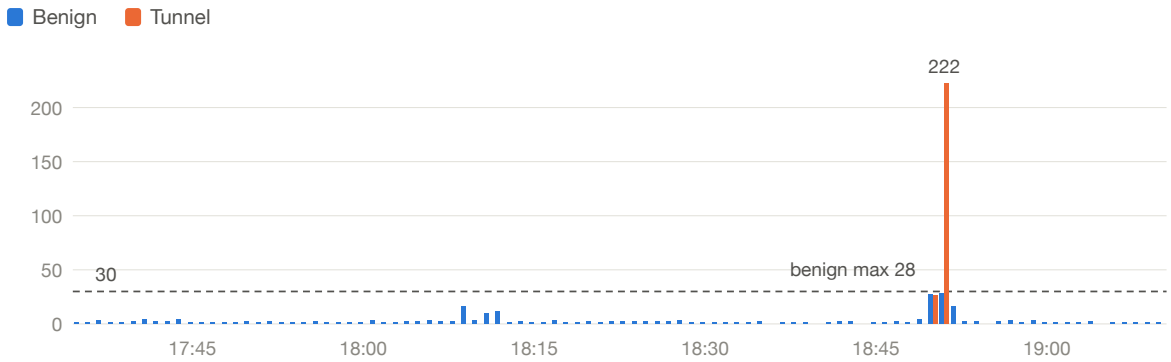
Keep one tunnel copy
Copy 2 matches copy 1 byte for byte. Keeping both doubles the positives and makes copy 2 look like repeated names.

Default keeps copy 2 (18:50)
It sits inside busy period 2, the harder test. Training can then include busy period 1.

Cut at 18:30
Middle of the quiet gap. Train: 1,592 benign rows, no tunnel. Test: 1,609 benign + 222 tunnel.

Strict variant: keep copy 1, cut at 18:04:36. Training is only 244 quiet rows, so it mostly shows what a load change does.

Why the context features work



Highest `uniq_names_60s` per minute (default run). The session straddles two minute bins, so the first shows 26 although the rolling count reaches 222.

The 8 features

Name shape (3)

`name_bytes` `longest_label` `entropy`

Data travels in the name: long, random-looking. Catches data and probe queries, not pings.

Domain context (4)

`uniq_names_60s` `uniq_ratio_60s` `norepeat_evidence` `log_cum_uniq`

Caching forces a new name per query. Catches the whole session.

Record type (1)

`data_qtype`

NULL or TXT flag. An on/off ablation switch.

Left out on purpose: IPs, ports, flags, TTL, EDNS0, rtt, NXDOMAIN, raw volume, domain names. They separate the classes only because of the lab.

Running it

```
python main.py                     default: copy 2, cut 18:30, 60 s window
python main.py --all --tests       default + strict variant, then the checks
python main.py --window 60 300 900 several window sizes (slow tunnels)
python main.py --session first     only copy 1
```

Start with `data/processed/features.csv`. Columns: `split` (train/test), `y` (label), 8 features. Features are raw, not scaled.

Suggested modelling

1. Fit Isolation Forest on the `forced` features, training rows only.
2. Threshold from the training scores (Tukey), no labels needed.
3. Evaluate on test: precision, recall, F1, recall per phase, false positives.
4. Compare with simple baselines, then stress tests: window size, strict variant, slow tunnel.

Limits to state

- One tunnel session, one tool, one benign client.
- The tunnel is loud, so results will look strong. Slow tunnels are the real test.
- The first dozen tunnel queries look benign. 6.5% of rows are tunnel, so precision is optimistic.

Generated 2026-11-11 from data/processed and the notebooks 02 and 03. See docs/data_dictionary.md and docs/handoff-2026-11-11-pipeline.md for details.